

Foundations United BridgeX Audit Summary

Prepared by Cyfrin

Version 2.0

Lead Auditors

Dacian

Jorge

April 1, 2026

Contents

1 About Cyfrin 2

2 About This Document 2

3 Disclaimer 2

4 Copyright and License 2

5 Audit Overview 3

6 Audit Scope 3

7 Deployment Bytecode Verification 4

8 Risk Classification 4

9 Protocol Summary 5

10 Attestation 5

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 About This Document

This document is a public summary of the security audit Cyfrin performed for Foundations United on BridgeX. It records the repositories and commits reviewed, the timeline, auditors, in-scope files, and the number and final status of findings by severity. It closes with Cyfrin's attestation of the review's scope and final remediation status.

It is deliberately abbreviated: the technical detail of individual findings is not reproduced here. The complete audit report, version 2.0 dated April 1, 2026, was delivered to Foundations United, who retain it as the full record of the engagement. This summary was issued on October 29, 2026.

3 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

4 Copyright and License

© 2026 Cyfrin Inc. All rights reserved. Material from Foundations United or third parties remains the property of its owners.

This summary may be shared only in complete and unmodified form, so that any copy can be checked against the hash in its attestation. Short quotations with attribution to Cyfrin are permitted, provided they do not misrepresent this summary's scope or results. Except as permitted by applicable law, no part of this summary may otherwise be copied, modified, adapted, sold or sublicensed, or used to create derivative works, without Cyfrin's prior written permission.

Cyfrin and the Cyfrin logo are trademarks of Cyfrin Inc. and may not be used to suggest that Cyfrin has reviewed or endorses anything outside this summary's scope, or to misrepresent its results.

5 Audit Overview

Over the course of 5 days, the Cyfrin team conducted an audit on the BridgeX code provided by Foundations United. The findings consist of 1 Medium & 7 Low severity issues with the remainder being informational and gas optimizations.

Summary

Project Name	BridgeX
Repository	bridge-x-contracts
Commit	406c951d3b3e...
Fix Commit	8153e1ee1002...
Audit Timeline	Mar 3rd - Mar 9th, 2026
Methods	Manual Review, Formal Verification

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	1	1	0
Low Risk	7	4	3
Informational	10	8	2
Gas Optimizations	9	8	1
Total	27	21	6

6 Audit Scope

The audit scope was limited to:

```
src/PrivateChainBridge.sol
src/PublicBridge.sol
src/Token.sol
```

7 Deployment Bytecode Verification

At the conclusion of the audit, after all mitigations were verified, Cyfrin was asked to verify that at the final mitigation commit 8153e1e the locally compiled bytecode matched the bytecode deployed on-chain. The following reproducible process was used.

First, prepare the repository:

```
gh repo clone NerdUnited-NodeGovernance/bridge-x-contracts
cd bridge-x-contracts
git checkout 8153e1ee1002cb440cf165a73adc3177289e27c4
forge install
forge build
forge test
```

Then, for each contract, fetch the deployed bytecode with `eth_getCode` and compare it with the `deployedBytecode` of the local Foundry build artifact (`out/Token.sol/Token.json`, `out/PublicBridge.sol/PublicBridge.json`), after stripping the CBOR metadata and ignoring the positions where the local bytecode holds zero bytes for immutable values:

- `Token` (Ethereum): 0x1ae1a1130e581240a9ed81355312fdb2e741f100
- `PublicBridge` (Ethereum): 0x544918caf667b200936718e51094800a959e7ffc

Both comparisons succeeded. The only differences were at known immutable positions: for `Token`, `decimals` (18) and `maxSupply` (50,000,000,000 × 10^{18}); for `PublicBridge`, the `token` immutable (the `Token` address above, 5 references) and the `vault` immutable (5 references), which holds the `TokenVault` at `0x9885ee74cb9ebdd2835ca2867c1a2877022c6dba` deployed by the `PublicBridge` constructor.

The client confirmed that the same contracts were deployed to BSC and Base mainnets at the same addresses. Cyfrin repeated the extraction and comparison for `Token` and `PublicBridge` on both chains using public RPC endpoints (`https://bsc-dataseed.binance.org` and `https://mainnet.base.org`); every comparison succeeded with the same immutable differences as on Ethereum mainnet.

All deployments across Ethereum, BSC, and Base mainnets are byte-for-byte identical (including constructor arguments and immutable values), confirming the same audited code is deployed on all three chains.

8 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

9 Protocol Summary

BridgeX is a bi-directional bridge that wraps native gas tokens on private GoQuorum-based blockchains ("brand chains") into ERC20 representations on public EVM chains (Ethereum, BSC, and Base). Users earn gas tokens on the private chain, then bridge (wrap) them to public chains where the newly minted ERC20 supply can be used to seed DEX/CEX liquidity pools.

The system consists of three contracts:

- `PrivateChainBridge` — Deployed on the private GoQuorum chain. Handles locking and releasing of native gas tokens via a `TokenVault`. When bridging outbound (private to public), users lock native tokens; when bridging inbound (public to private), native tokens are released from the vault.
- `PublicBridge` — Deployed on each public EVM chain. Handles locking and releasing of `Token` ERC20 tokens via a `TokenVault`. When bridging outbound (public to private), users lock `Token` ERC20 tokens in the vault; when bridging inbound (private to public), tokens are released from the vault or minted if the vault has insufficient balance.
- `Token` — A standard ERC20 contract deployed on each public chain. Represents the wrapped form of the private chain's native gas token, with new supply minted by the bridge up to a hard `maxSupply` cap.

Relayer nodes monitor `TokensLocked` events on the source chain and sign the corresponding release on the destination chain. Releases require users to pre-pay a `releaseFee` in the destination chain's native token which is distributed among relayers. Users who have not pre-paid receive pending releases that are automatically processed upon fee payment.

A `bridgeFee` denominated in the bridged token is deducted on the source chain during locking and sent to a fee receiver address. A `DestinationChain` enum with reserved values and a `chainIds` mapping allow extending to additional chains without redeployment.

10 Attestation

Cyfrin attests that it performed a security review of the BridgeX code provided by Foundations United.

The repositories, commits, review period and findings with their final status are in the Summary and Issues Found tables in Section 5, and the in-scope files in Section 6. This attestation is limited to the documented scope, audited commits and mitigations specifically reviewed by Cyfrin.

Cyfrin publishes this document and its SHA-256 hash at https://github.com/Cyfrin/cyfrin-audit-reports. To check a copy on macOS or Linux, run the following in the folder that contains it; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name. Before running it, check that the URL is Cyfrin's repository.

```
U=https://github.com/Cyfrin/cyfrin-audit-reports/raw/main/reports/sha256
N=2026-04-01-cyfrin-bridgex-v2.0.pdf
F=$N
H=$(curl -fsSL $U/$N.sha256 | cut -d' ' -f1)
echo "$H *$F" | shasum -a 256 -c
```

Cyfrin's full audit report with all findings provided to Foundations United has the SHA-256 hash and file name below. To check a copy of it, first check this document as above, then run the following in the folder that contains the full report; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name.

```
H=cb706d9f22367663f1d05d743e0400381a878686a71350fa00e92d89143eea77
F=2026-04-01-cyfrin-bridgex-v2.0-full.pdf
echo "$H *$F" | shasum -a 256 -c
```

On Linux, `sha256sum -c` can be used in place of `shasum -a 256 -c`. On Windows, run `Get-FileHash <file> -Algorithm SHA256` in PowerShell and compare the result with the hash.